

# CS101: Computer Programming



## Lec 01: Introduction to Computing

Dr. Soumitra Ghosh  
Assistant Professor, CSE  
IIT (BHU) Varanasi

[soumitra.cse@iitbhu.ac.in](mailto:soumitra.cse@iitbhu.ac.in)

# How the courses are run? Policies, Guidelines..

- There are 8 TAs who will primarily help you in the Laboratory sessions and Tutorial Classes.
- **Questions in the examinations** will be from what will be taught in the classes.
- Ask your doubts and questions to me. Do not discuss in the class with your friends.
- **Attendance for Laboratory sessions is important** and will be counted.
- **Grading** will be strictly according to **your performance, attendance**, etc.

# Course Objectives

- To give basic programming skills.
- To give conceptual understanding of computers and their working.
- At the end of the course the student is expected to write a moderately complex
- C program.

# Evaluation Criteria

## Theory

- Mid Semester Exam: 30%
- End Semester Exam: 40%
- Assignments / Quizzes: 10%

## Laboratory:

- Assignments + Viva: 20%

# Laboratory Part

- Use only *Linux/Ubuntu/Fedora* Operating System.  
[ Do not use *Microsoft windows* ! ]
- Use ANSI C (Standard C).
- Use *vi/nano* editor for creating your files.
- Brief about the 1<sup>st</sup> exercise will be given towards the end of this class.

# Reference Books and Course Page

You may follow any standard ANSI C book.

- C Programming: A Modern Approach by K. N. King
- Let us C, Yashavant Kanetkar
- **Programming in ANSI C, E. Balagurusamy**
- Programming With C pdf – Byron Gottfried
- Brian W. Kernighan, Dennis M. Ritchie. The C Programming Language

Initially Lecture slides and materials will be shared via Gdrive

In due time, a dedicated course page will be designed for materials dissemination.



# Learning Objectives

**In this lecture you will learn about:**

- Computer
- Characteristic features of computers
- Stored Program Concept
  - von Neumann bottleneck
  - Language Evolution
  - Compiler - Interpreter

# Computer

- The word computer comes from the word “**compute**”, which means, “to calculate”
- Thereby, a computer is an electronic device that can perform arithmetic operations at high speed
- A computer is also called a *data processor* because it can store, process, and retrieve data whenever desired



# Computer

- The word computer comes from the word “compute”, which means, “to calculate”
- Thereby, a computer is an electronic device that can perform arithmetic operations at high speed
- A computer is also called a *data processor* because it can store, process, and retrieve data whenever desired

## Characteristics

? ? ? ? ? ? ? ?

# Computer

- The word computer comes from the word “**compute**”, which means, “to calculate”
- Thereby, a computer is an electronic device that can perform arithmetic operations at high speed
- A computer is also called a *data processor* because it can store, process, and retrieve data whenever desired

## Characteristics

Automatic | Speed | Accuracy | Diligence | No IQ | No Feelings | Versatility | Power of Remembering

# Stored Program Concept

**Key Idea:** Both the program instructions and the data are stored together in the computer's main memory.

- CPU fetches instructions one by one from memory, decodes them, executes them, and stores the results back in memory.
- Allows:
  - Easy modification of programs
  - Reusability
  - Automatic execution without rewiring hardware

**Before Stored Program ?????**

# Before Stored Program

Programs were:

- Hard-wired, connected using switches and cables
- Difficult to modify and time-consuming to reprogram
- Every new task required rewiring, restructuring and/or redesigning the fixed-program machine
- Also known as **Fixed Programs**
  - *Example: Calculator*
    - Can do basic mathematics, but cannot be used as word processor or gaming console
  - *ENIAC*: It could take 3 weeks to set up a program and get it working



# Technicians connecting wires of ENIAC

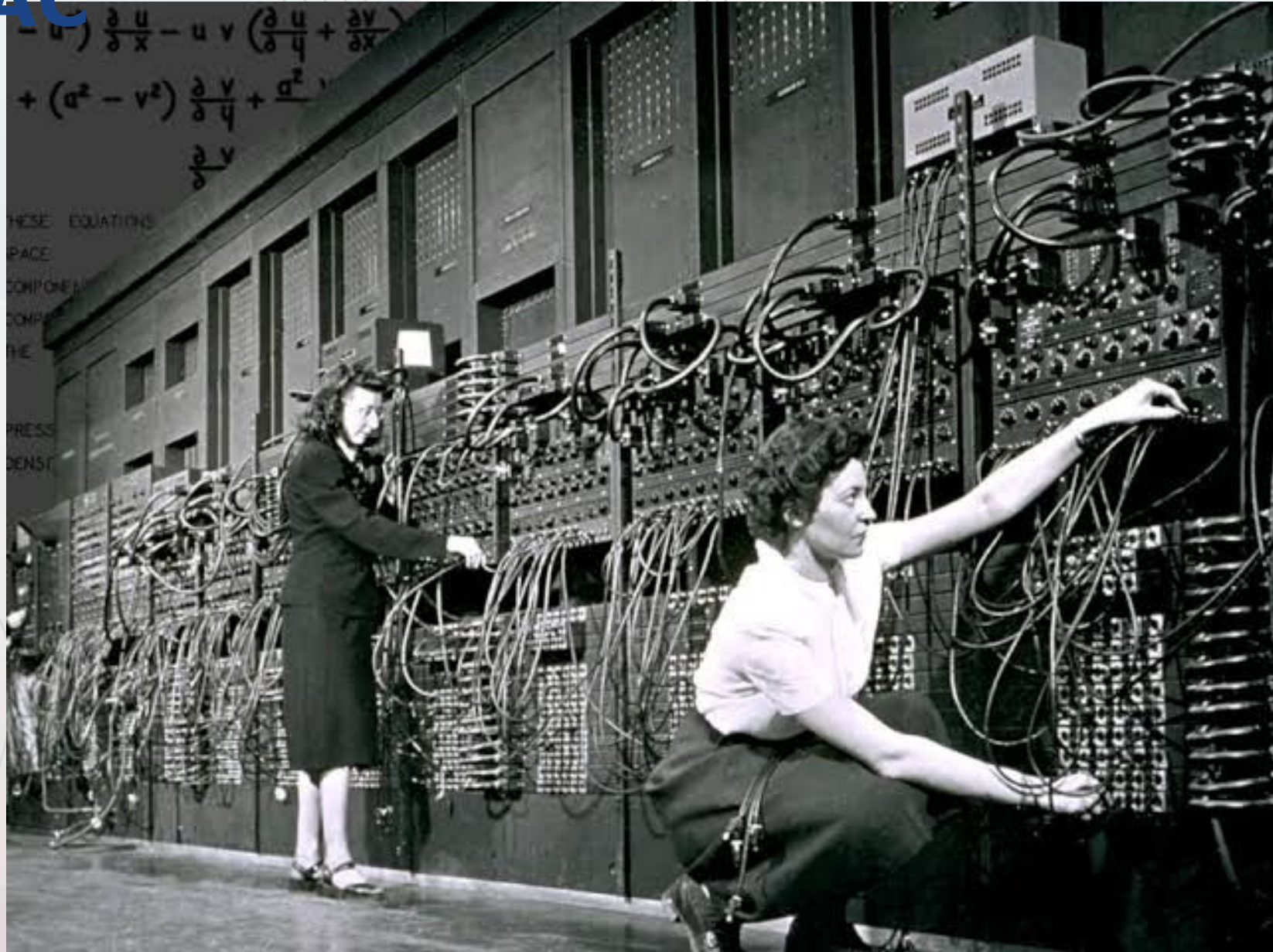


Photo source: <https://cse.engin.umich.edu/about/history/eniac-display/>

# History of the Stored Program Concept

## ★ Early Computers (1940s)

- ENIAC, Colossus
- Characteristics:
  - No stored program; Programs entered manually; Wiring changes required for every task

## ★ John von Neumann (1945)

- Proposed the Stored Program Architecture in the report:
  - *First Draft of a Report on the EDVAC*
  - Main idea:
    - Store instructions and data together in memory.
    - CPU executes instructions sequentially.

## ★ First Stored Program Computers

- Manchester Baby (1948), EDSAC (1949), EDVAC, IAS Computer
- These became the basis of modern computers.



First Draft of a Report  
on the EDVAC

by

John von Neumann



# (John) von Neumann

also known as *Princeton Architecture*

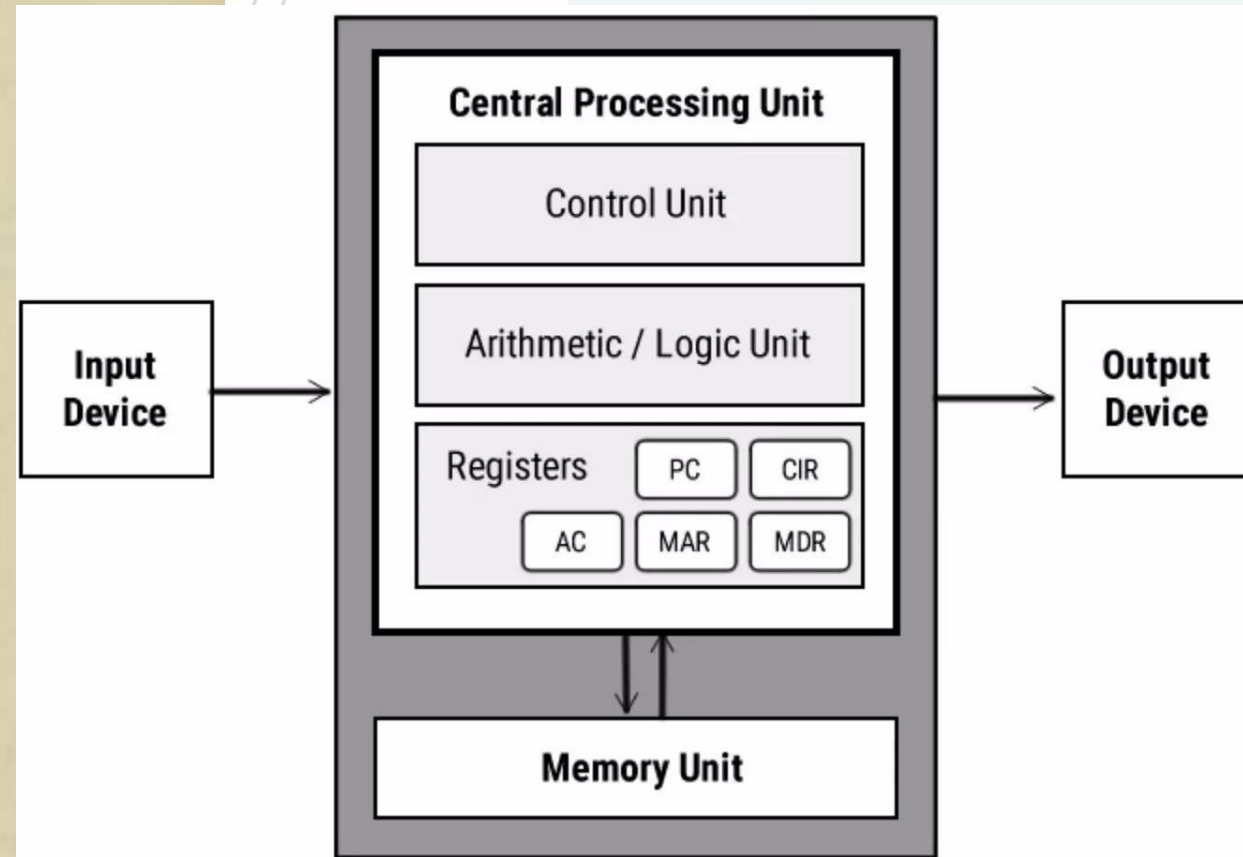
## Architecture

Contract No. W-670-ORD-4926

Between the  
United States Army Ordnance Department  
and the  
University of Pennsylvania

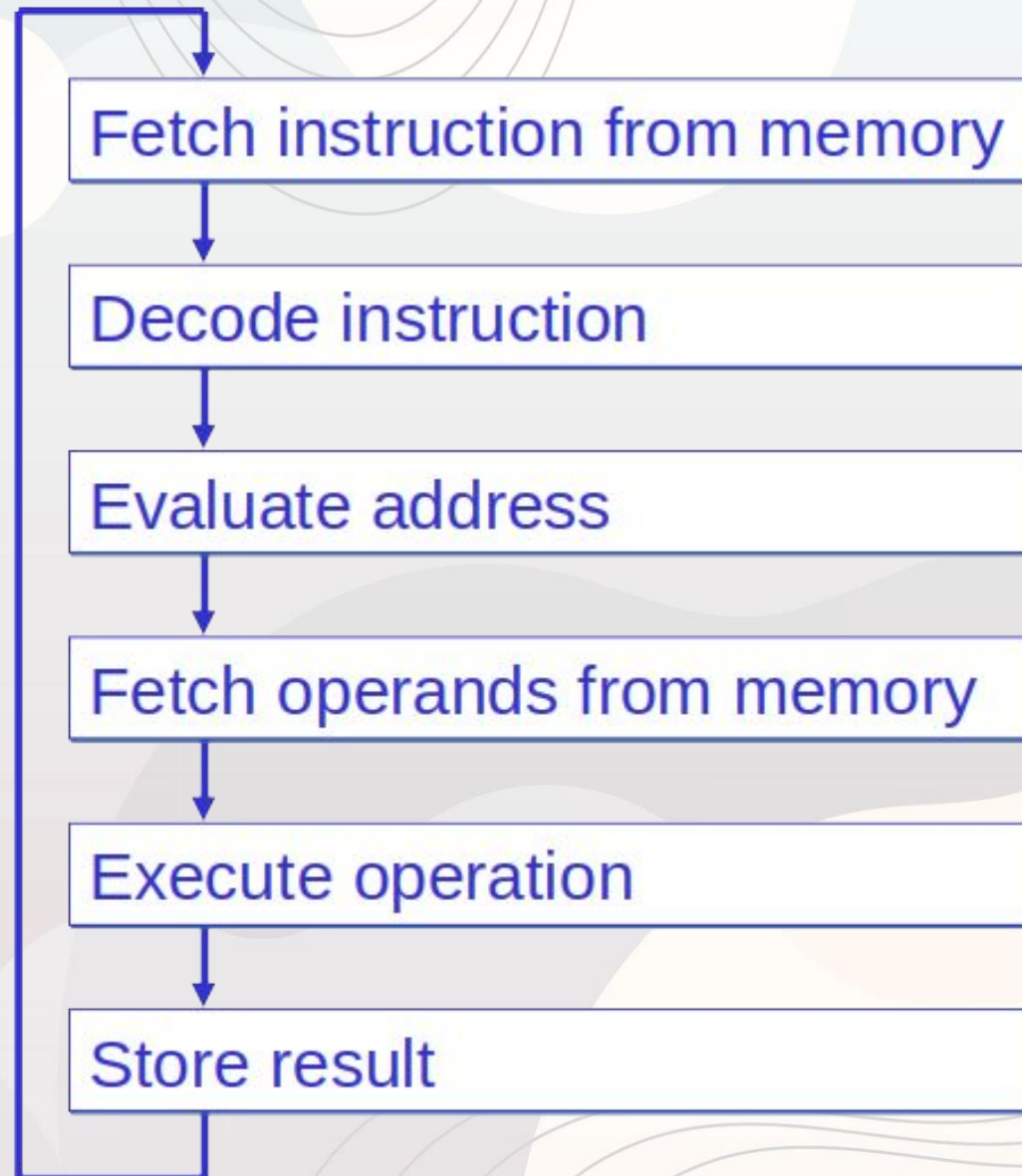
Moore School of Electrical Engineering  
University of Pennsylvania

June 30, 1945



# Working of Von Neumann Model

## Instruction Processing



# Stored Program Concept: Advantages & Disadvantages

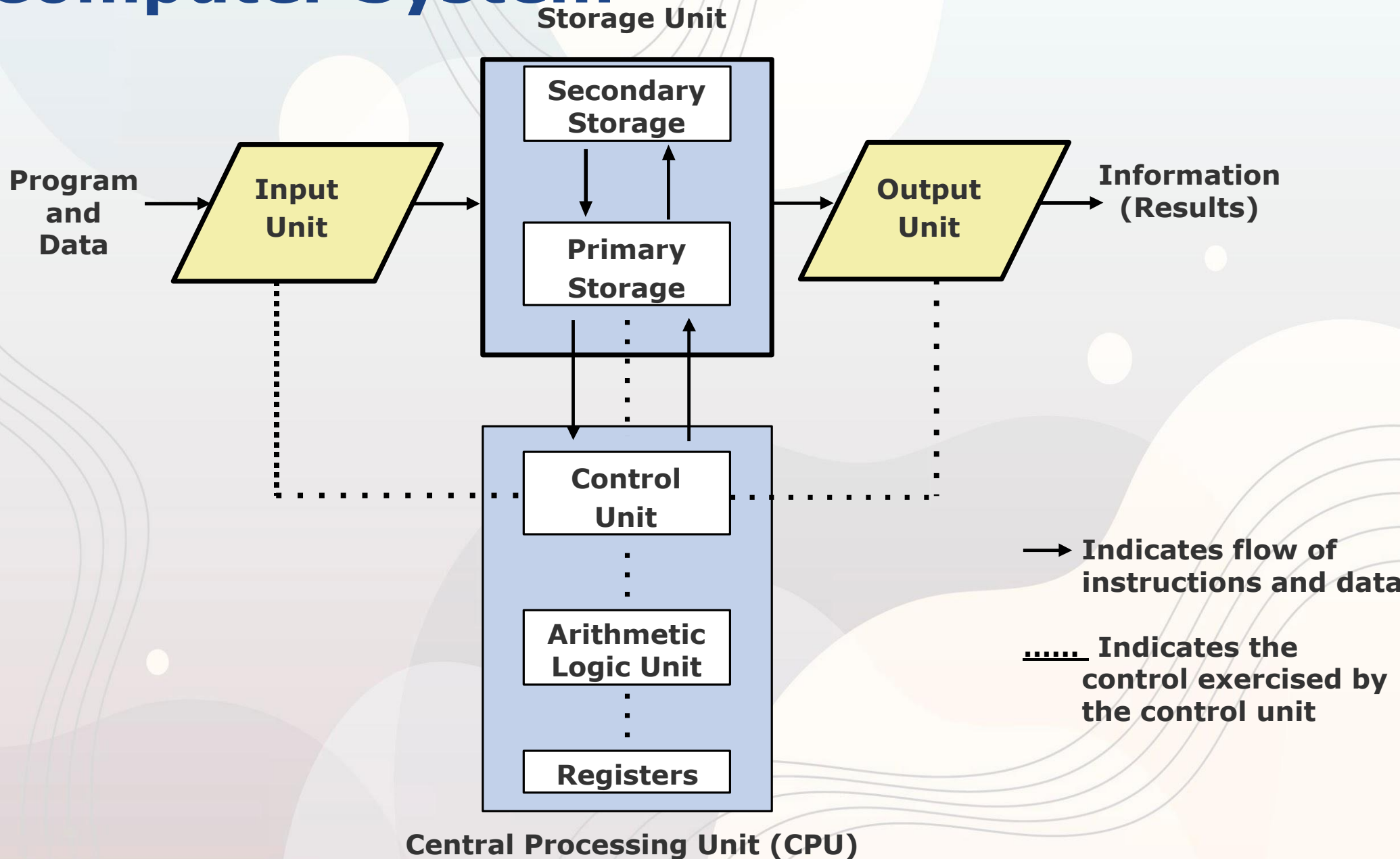
## Advantages

- ✓ Programs and data can be stored together in memory.
- ✓ Easy to modify or update programs without changing hardware.
- ✓ Supports automatic execution using the Fetch–Decode–Execute cycle.
- ✓ Same hardware can run different applications by simply changing the program.
- ✓ Simplifies programming and enables general-purpose computing.

## Disadvantages

- ✗ Instructions and data share the same memory, causing contention.
- ✗ Suffers from the Von Neumann Bottleneck due to limited CPU–memory bandwidth.
- ✗ Sequential instruction execution limits parallelism.
- ✗ Performance depends heavily on memory access speed.
- ✗ Not ideal for modern AI and data-intensive applications requiring massive parallel processing.

# Basic Organization of a Computer System



# The Five Basic Operations of a Computer System

- **Inputting.** The process of entering data and instructions into the computer system
- **Storing.** Saving data and instructions to make them readily available for initial or additional processing whenever required
- **Processing.** Performing arithmetic operations (add, subtract, multiply, divide, etc.) or logical operations (comparisons like equal to, less than, greater than, etc.) on data to convert them into useful information
- **Outputting.** The process of producing useful information or results for the user such as a printed report or visual display
- **Controlling.** Directing the manner and sequence in which all of the above operations are performed



# Input Unit

1. Accepts (or reads) instructions and data from outside world
2. Converts these instructions and data in computer acceptable form
3. Supplies the converted instructions and data to the computer system for further processing

# Output Unit

1. Accepts the results produced by the computer, which are in coded form and hence, cannot be easily understood by us
2. Converts these coded results to human acceptable (readable) form
3. Supplies the converted results to outside world

# Storage Unit

Holds (or stores) the following :

1. Data and instructions required for processing (received from input devices)
2. Intermediate results of processing
3. Final results of processing, before they are released to an output device

## Primary storage

- Hold running program instructions
- Hold data, intermediate results, and results of ongoing job(s)
- Fast in operation
- Small Capacity
- Expensive and Volatile

## Secondary storage

- Hold stored program instructions
- Hold data and information of stored jobs
- Slower than primary storage
- Large Capacity
- Lot cheaper than primary storage
- Retains data even without power

# Central Processing Unit (CPU)

Place where the actual executions of instructions takes place during processing operation

Manages and coordinates the operations of all other components of the computer system

**Arithmetic  
Logic Unit  
(ALU)**

+

**Control  
Unit (CU)**

=

**Central  
Processing  
Unit (CPU)**

- It is the brain of a computer system
- It is responsible for controlling the operations of all other units of a computer system

# CPU Registers

| Register                                                                                         | Purpose                                                                                     | Example                                                                                |
|--------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------|
| <b>AC</b><br><b>Accumulator Register</b>                                                         | Stores intermediate arithmetic and logical results produced by the ALU.                     | $10 + 20 = 30 \rightarrow$ Result stored in AC                                         |
| <b>MAR</b><br><b>Memory Address Register</b>                                                     | Holds the address of the memory location to be read from or written to.                     | To read data from address 2050, MAR = 2050                                             |
| <b>MDR</b><br><b>Memory Data Register</b>                                                        | Holds the data being transferred between CPU and main memory.                               | Data "75" read from memory is stored in MDR                                            |
| <b>PC</b><br><b>Program Counter</b>                                                              | Stores the address of the next instruction to be executed. Updated after every instruction. | If current instruction is at 1000, PC may point to 1004 (assuming 4-byte instructions) |
| <b>CIR</b><br><b>Current Instruction Register</b> <i>(also called Instruction Register - IR)</i> | Holds the instruction currently being decoded and executed by the CPU.                      | Stores instruction ADD R1, R2 during execution                                         |

# John von Neumann **Bottleneck**

The **limited data transfer speed** between the CPU and memory is called the Von Neumann Bottleneck.

## Why does it occur?

- CPU speed  $\gggg$  Memory speed
- Only one bus is available.
- Instructions and data compete for access.

## Consequences

- CPU idle time and increased execution time
- Lower throughput and reduced performance



# John von Neumann **Bottleneck**

The limited data transfer speed between the CPU and memory is called the Von Neumann Bottleneck.

## Why does it occur?

- CPU speed  $\gggg$  Memory speed
- Only one bus is available.
- Instructions and data compete for access.

## Consequences

- CPU idle time and increased execution time
- Lower throughput and reduced performance

**Overcoming  
the Von  
Neumann  
Bottleneck**



??

# John von Neumann **Bottleneck**

The limited data transfer speed between the CPU and memory is called the Von Neumann Bottleneck.

## Why does it occur?

- CPU speed  $\gggg$  Memory speed
- Only one bus is available.
- Instructions and data compete for access.

## Consequences

- CPU idle time and increased execution time
- Lower throughput and reduced performance

## Overcoming the Von Neumann Bottleneck

### Improving Von Neumann

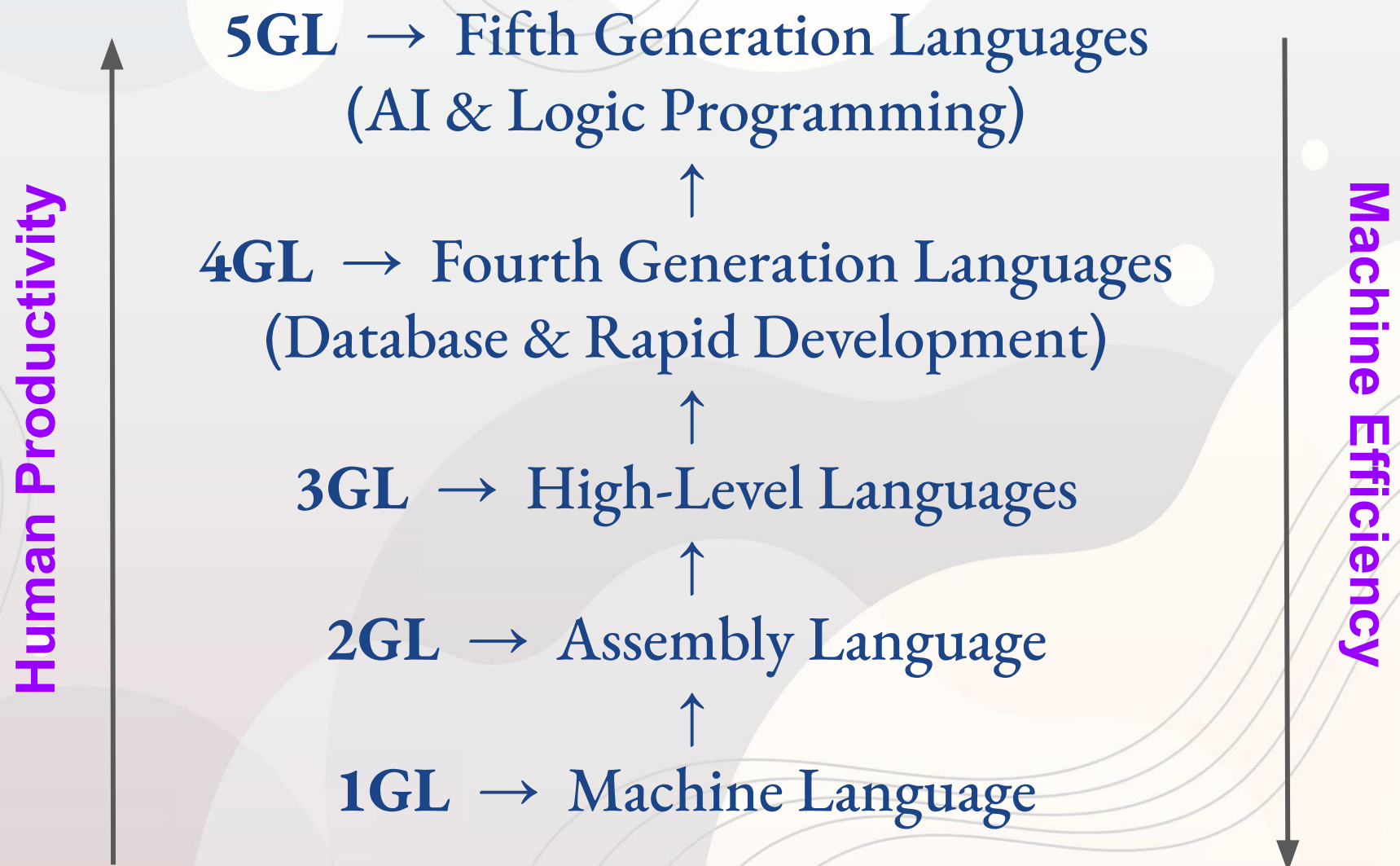
Cache Memory  
Pipelining  
Superscalar Execution  
Branch Prediction  
Multi-core Processors

### Beyond Von Neumann

Harvard Architecture  
Dataflow Architecture  
Neuromorphic Computing  
Processing-in-Memory  
Quantum Computing

# Hierarchy of Computer Languages

A computer understands only machine language (binary). To make programming easier for humans, several levels of programming languages have been developed.



# First Generation Language (1GL) – Machine Language

The lowest-level programming language consisting entirely of binary digits (0s and 1s) directly understood by the CPU.

## Example

10110000 01100001

00000101

11000011

## Characteristics

- Executed directly by the processor
- Fastest execution
- Machine-dependent
- Difficult to write and debug

## Advantages

- ✓ No translator required
- ✓ Highest execution speed

## Disadvantages

- ✗ Difficult to understand
- ✗ Error-prone
- ✗ Not portable

# Second Generation Language (2GL) – Assembly Language

Uses mnemonic instructions instead of binary codes. Requires an Assembler to convert into machine language.

## Example

MOV AX, 10

ADD AX, 5

SUB AX, 2

## Characteristics

- Easier than machine language
- Hardware-specific
- Faster than high-level languages
- Requires knowledge of processor architecture

## Advantages

- ✓ Easy to read compared to binary
- ✓ Efficient use of hardware

## Disadvantages

- ✗ Machine dependent
- ✗ Difficult for large applications



# Third Generation Language (3GL) – High-Level Language

- Designed to resemble human language and mathematical notation, making programming easier and portable.
- Requires a Compiler or Interpreter.

## Examples

- C, C++, Java, Python, Pascal, FORTRAN

### Example (C)

```
#include <stdio.h>
int main() {
    int a = 10, b = 20;
    printf("%d", a + b);
    return 0;
}
```

## Advantages

- ✓ Easy to learn
- ✓ Portable
- ✓ Faster development
- ✓ Easier debugging

# 4GL and 5GL

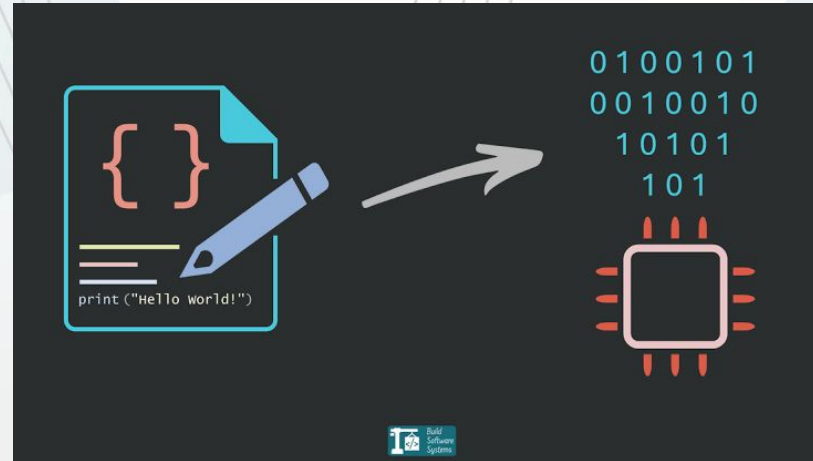
| Fourth-Generation Language (4GL)                                                                                                                                                                                  | Fifth-Generation Language (5GL)                                                                                                                                                                                                                      |
|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| High-level, declarative languages designed to simplify application and database development by specifying what needs to be done rather than how to do it.                                                         | Logic- and constraint-based languages designed primarily for Artificial Intelligence (AI), knowledge representation, and expert systems, where programmers specify the problem and the system determines the solution.                               |
| <ul style="list-style-type: none"><li>• Very high-level and user-friendly</li><li>• Fewer lines of code</li><li>• Rapid application development</li><li>• Commonly used for databases and data analysis</li></ul> | <ul style="list-style-type: none"><li>• Based on logic and inference</li><li>• Supports AI and expert systems</li><li>• Emphasizes problem specification over algorithms</li><li>• Suitable for reasoning and knowledge-based applications</li></ul> |
| <b>Example (SQL):</b><br>SELECT Name, Salary<br>FROM Employee<br>WHERE Salary > 50000;                                                                                                                            | <b>Example (Prolog):</b><br>parent(john,mary).<br>parent(mary,alice).<br>grandparent(X,Y):-<br>parent(X,Z),<br>parent(Z,Y).                                                                                                                          |
| <b>Applications:</b> Database management, report generation, business intelligence, data analytics                                                                                                                | <b>Applications:</b> Artificial Intelligence, Expert Systems, Robotics, Natural Language Processing                                                                                                                                                  |

# Comparison of Computer Languages

| Feature                    | 1GL            | 2GL          | 3GL                  | 4GL             | 5GL                          |
|----------------------------|----------------|--------------|----------------------|-----------------|------------------------------|
| <i>Language</i>            | Machine        | Assembly     | High-Level           | Very High-Level | Logic/AI                     |
| <i>Representation</i>      | Binary         | Mnemonics    | English-like         | Declarative     | Logic-based                  |
| <i>Translator</i>          | None           | Assembler    | Compiler/Interpreter | Compiler        | Interpreter/Inference Engine |
| <i>Hardware Dependency</i> | High           | High         | Low                  | Very Low        | Very Low                     |
| <i>Ease of Programming</i> | Very Difficult | Difficult    | Easy                 | Easier          | Easiest                      |
| <i>Execution Speed</i>     | Highest        | High         | Moderate             | Moderate        | Lower                        |
| <i>Examples</i>            | Binary Code    | x86 Assembly | C, Java, Python      | SQL, MATLAB     | Prolog                       |

# Language Translators

Programmers write programs using assembly language or high-level languages.

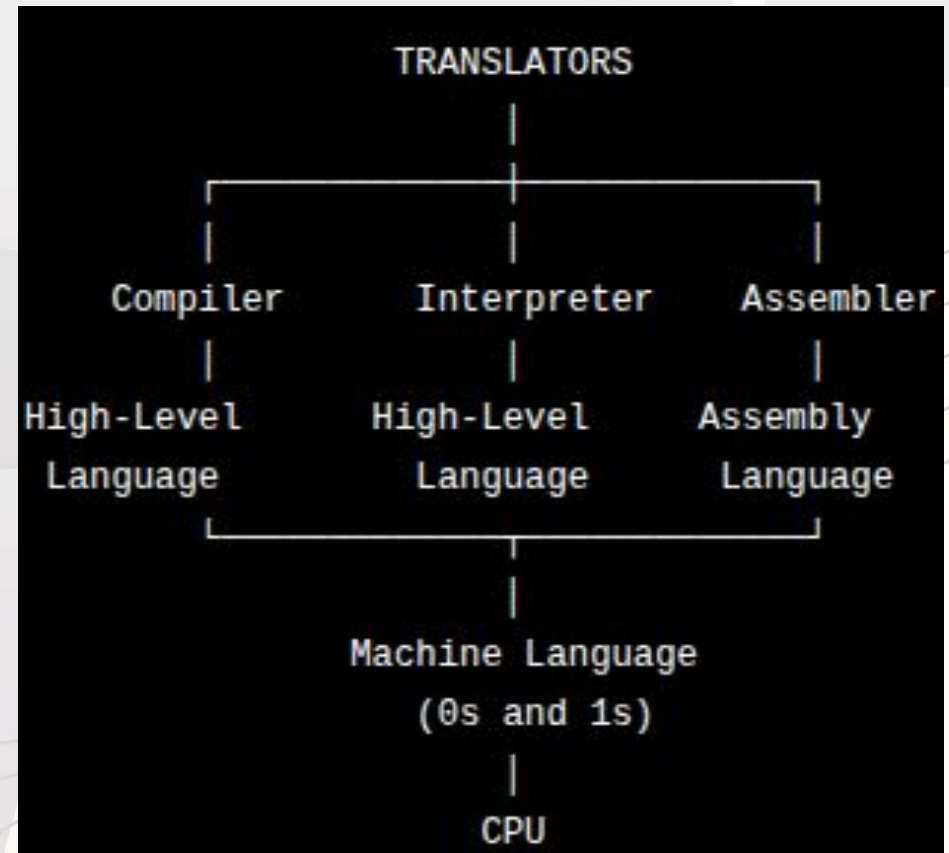


Computers understand only machine language (binary: 0s and 1s)

Language translator: System software that converts source code into machine code for executed by the CPU.

## Types of Language Translators

- Assembler
- Compiler
- Interpreter



# Assembler

- Converts assembly language into machine language.
- Uses **mnemonics**
  - MOV, ADD, SUB, INC, DEC, JMP
- CPU cannot understand these.
  - Assembler converts them into binary.

**MOV AX,5** (*Assembly Language*)



**Assembler**



**101100010101** (*Machine Language*)

**Assembly Language**



**Assembler**



**Machine Code**



**CPU**



# Compiler

A compiler converts the entire high-level program into machine code before execution.

**Entire Program >> Compiler >> Machine Code >> Execute**

Only after reading the complete program does it generate machine code.

## Example Languages

- C, C++, Go, Rust

## Advantages

- ✓ Faster execution
- ✓ Finds many errors together
- ✓ Generates executable file
- ✓ Better optimization

## Disadvantages

- Compilation takes time.
- Must recompile after every change.

## Compilation Process

**Source Program (.c)**



**Compiler**



**Object Code (.obj)**



**Linker**



**Executable (.exe)**

# Interpreter

Interpreter translates and executes one statement at a time.

Read Line 1 >> Translate >> Execute

Read Line 1 >> Translate >> Execute

Read Line 1 >> Translate >> Execute

**Example Languages:** Python, JavaScript, Ruby, MATLAB, Shell scripting

- **Advantages**

- a. Easy debugging
- b. No object code
- c. Immediate execution
- d. Good for testing

- **Disadvantages**

- a. Execution is slower.
- b. Every execution requires translation again.

## Example

```
print("Hello")  
print("World")
```

## Interpreter

Translate line 1

Execute

Translate line 2

Execute

# Compiler vs Interpreter

| Compiler                                                                   | Interpreter                                                |
|----------------------------------------------------------------------------|------------------------------------------------------------|
| Converts the entire source code into executable machine code.              | Translates and executes the source code line by line.      |
| Takes longer to analyze the source code, but program execution is faster.  | Analyzes quickly, but overall program execution is slower. |
| Reports errors after scanning the entire program, making debugging harder. | Reports errors as they occur, making debugging easier.     |
| Requires more memory to generate object code.                              | Requires less memory as no object code is generated.       |
| Generates intermediate object code.                                        | No intermediate object code is generated.                  |
| More useful for security purpose.                                          | Little vulnerable in case of security.                     |
| Examples: C, C++, C#                                                       | Examples: Python, Perl, JavaScript, Ruby.                  |

# Your first lab exercise

- Learn basic commands to work with a terminal (preferably in Linux environment).
- You can use VS Code (for Linux, learn vi editor)
- Create a file named `first.c` whose contents should be:
- ```
#include<stdio.h>
int main()
{
    printf(" Hello IIT(BHU)");
    return 0;
}
```

# Commands

- After you create the file whose name is *first.c* you need to compile this file to get executable file.
- At the command prompt do
  - `$ cc first.c`
- If the compilation is successful this gives a file called *a.out*
  - `$ ./a.out`
- The above command executes it.



# References

- <https://voccomputerscience.orgfree.com/ComputerFundamentals/>
- <https://www.geeksforgeeks.org/computer-science-fundamentals/language-processors-assembler-compiler-and-interpreter/>